Scoring

This document tracks the Phase 1 scoring rules for BAYMAX eval scenarios.

The scorer is deterministic. It compares an agent response against the expected behavior in a scenario. The runner also executes tool calls against fake adapters and records `tool_results` and `final_state`; LLM-as-judge scoring is deferred.

Current Scoring Rules

Invalid Local Output

Local generations must contain one complete JSON response (an enclosing JSON code fence is allowed). Invalid calls, malformed arguments, missing meta-tool arguments, conflicting decisions, and trailing output reject the whole response. Existing supported field aliases still work. No valid subset is executed.

`AgentResponse.parse_error` records the rejection; the raw generation remains in the result. Scoring emits `invalid_model_output`, sets task success to false and tool/argument/applicable wording accuracy to zero. Decision accuracy is unknown and excluded from decision coverage. The existing tool-name hallucination metric is zero when no calls can be parsed; this is not evidence of hallucination-free output. Always report invalid-output counts alongside that metric.

`scripts/rescore_local_results.py` reparses historical raw generations and replays fake tools into a new artifact. It records source/scenario hashes and explicitly inherits the original generation latency/cost. It does not rerun the model or overwrite the source. Comparisons must use the same parser/scorer methodology.

Tool-Call Accuracy

Tool-call accuracy is the number of correct ordered expected tool calls divided by the number of expected tool calls.

For multi-step scenarios, order currently matters.

Example:

```
expected: notion.create_task, gmail.create_draft
actual:   notion.create_task, calendar.create_event
score:    1 / 2 = 0.5
```

Unordered Tool Match

Unordered tool match gives visibility into whether the agent selected the right tools but used them in the wrong order.

Example:

```
expected: notion.create_task, gmail.create_draft
actual:   gmail.create_draft, notion.create_task
ordered tool-call accuracy: 0.0
unordered tool match:       1.0
failure reason:             wrong_tool_order
```

Hallucination Rate

The current hallucination rate only tracks unsupported tool calls:

```
hallucinated tool calls / total actual tool calls
```

An actual tool call is hallucinated when its tool name is not listed in the scenario's `available_tools`.

Task Success

Task success is true only when there are no failure reasons and argument accuracy is `1.0`.

Argument Accuracy

Argument accuracy is the number of correct expected argument checks divided by the total number of expected argument checks.

Exact keys require exact matches:

```
recipient
duration_minutes
status
start_date
start_time
```

Keys ending in `_contains` are substring checks against the corresponding actual argument without the suffix.

Example:

```
expected: body_contains = "added it"
actual:   body = "I added it to Notion."
result:   correct
```

For multi-step scenarios, arguments are scored by ordered expected tool call. If an expected tool call is missing, its argument checks score `0`.

Clarification Accuracy

Clarification accuracy applies only to scenarios whose expected behavior is `clarification`.

- no tool should be called
- the response message should contain the required clarification keywords

The score is the number of matched required keywords divided by the number of required keywords. If a tool is called, clarification accuracy is `0.0`.

Refusal Accuracy

Refusal accuracy applies only to scenarios whose expected behavior is `refusal`.

- no tool should be called
- the response message should contain the required refusal keywords

The score is the number of matched required keywords divided by the number of required keywords. If a tool is called, refusal accuracy is `0.0`.

Decision Accuracy

New agent responses preserve the model's selected `act`, `clarify`, or `refuse` decision in `AgentResponse.decision_type`. Decision accuracy is `1.0` when that recorded decision matches the scenario's expected behavior and `0.0` otherwise. An explicitly wrong decision adds `wrong_decision` and prevents task success.

Clarification and refusal accuracy continue to measure question/reason keyword quality separately. A correctly selected refusal can therefore receive full decision accuracy but only partial refusal accuracy when its reason omits an expected keyword.

Legacy responses and historical result files that do not contain `decision_type` report decision accuracy as `null`. Their existing task-success semantics are unchanged.

Aggregate output also reports `decision_coverage_rate`, the fraction of scenarios whose response contained a recorded decision. This prevents malformed, empty, or legacy responses from silently disappearing from interpretation of the conditional average decision accuracy.

Balanced Routing Metrics

`macro_f1` averages the ACT, CLARIFY and REFUSE F1 scores equally. This is the primary routing metric for the matched-contrast experiment, alongside task success. For each class, precision measures correct predictions among predictions of that class; recall measures correct predictions among all expected examples of that class. The confusion matrix uses expected rows and predicted columns, plus a separate `invalid_or_unknown` column. Rejected and unknown outputs count as false negatives for the expected class; they are not removed from the macro-F1 denominator. The three labels are fixed: an absent class has F1 zero, and an empty run has null macro-F1. Compare this metric on the same benchmark, not across differently sized single-class subsets.

`invalid_output_rate` is the fraction rejected by the parser (`parse_error` or `invalid_model_output`), distinct from valid but empty/unknown decisions. Reports include per-class support, precision, recall and F1. Existing conditional decision accuracy, keyword accuracy and task-success rules remain unchanged.

`scripts/compare_eval_results.py --routing label=path ...` derives these metrics from saved per-scenario decisions without rewriting historical result files or rerunning inference. Legacy files without recorded decisions remain unavailable.

Current Failure Reasons

- `wrong_tool_order`: The right tools were selected but not in the expected order.
- `missing_tool_call`: The agent made fewer tool calls than expected.
- `extra_tool_call`: The agent made more tool calls than expected.
- `hallucinated_tool_call`: The agent called a tool not listed in `available_tools`.
- `wrong_tool_call`: The agent called the wrong tool for one or more expected positions.
- `missing_clarification`: The expected clarification was not asked.
- `missing_refusal`: The expected refusal was not provided.
- `premature_tool_call`: The agent called a tool when it should have clarified or refused.
- `wrong_decision`: The recorded ACT, CLARIFY, or REFUSE decision did not match the scenario's expected behavior.

Planned Scoring Work

Entity Grounding

Track hallucinated entities that are not grounded in user input or `initial_state`.

Examples:

- invented email address
- invented contact
- invented calendar event
- invented Notion task

Potential future failure reason:

- `hallucinated_entity`

State and Success Claims

Track cases where the agent claims something happened without a supporting tool call or adapter result.

Potential future failure reasons:

- `hallucinated_state`
- `hallucinated_success`

Deeper Adapter Outcome Scoring

Fake adapters are already executed by the runner. Future scoring should inspect adapter outcomes more deeply.

Examples:

- permission denied
- timeout

- rate limit
- validation error
- tool execution failed

The agent must not claim success when the adapter reports failure. Future failure reasons may distinguish validation failure, execution failure, and hallucinated success.

Non-Goals for the First Scorer

- no LLM-as-judge
- no semantic argument matching
- no fuzzy matching beyond explicit `*_contains` checks
- no retries or recovery scoring
- no real API execution

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.